Model Training Approach & Technical Report

Summary

This section documents the development phase of the Bank Transaction Fraud Detection system (Group B, Member 2). Using the preprocessed datasets (train_processed.csv and test_processed.csv), we implemented and compared four foundational algorithms: Logistic Regression, Decision Tree, Random Forest, and XGBoost. This report details the core design decisions, modeling configurations, and performance rationales of the training setup.

1. Handling the Training-to-Validation Discrepancy

A critical challenge in fraud detection modeling is the difference in class distribution between training data and real-world application:

- **The Training Space:** Engineered with a synthetic 5:1 oversampling ratio (~16.67% fraud rate) to provide enough fraud patterns for mathematical optimization.
- **The Validation Space:** Untouched 19-feature test set maintaining the natural 5.05% imbalance.

Because the model must deploy into a real-world environment where fraud is rare, all final validation evaluations are run strictly on the untouched test set. This ensures that the evaluation metrics accurately reflect actual business operations.

2. Model Implementations & Algorithmic Rationales

Each model was chosen to represent a distinct family of mathematical modeling, providing diverse approaches to the feature space.

A. Logistic Regression (Baseline Linear Model):

Logistic regression uses a logistic (sigmoid) function to model a binary dependency.
Parameters set are class_weight='balanced' and max_iter=1000.
Because standard numeric variables were pre-scaled, aligning perfectly with linear regression assumptions. class_weight='balanced' acts as a cost-sensitive learning rate, penalizing misclassified fraud events more heavily than legitimate ones to offset residual class discrepancies.

B. Decision Tree (Non-Linear Rule Base):

Decision trees split data recursively based on feature thresholds that maximize information gain (minimizing entropy).
Parameters were set as max_depth=10 and class_weight='balanced'.
Without boundaries, decision trees will overfit the oversampled minority cases. Limiting depth to 10 forces the model to generalize patterns rather than memorizing individual transactions, while the weight parameter balances the decision boundary.

C. Random Forest (Bagged Ensemble):

Random forest trains an ensemble of independent decision trees over random subsets of the data and features, averaging their outputs.

The parameters were configured with 100 estimators (n_estimators=100) and class_weight='balanced_subsample' because balanced_subsample is more effective for imbalanced data. Instead of calculating weights globally, it adjusts class weights dynamically inside each bootstrap sample used to build individual trees. This ensures minority fraud instances remain influential across every tree.

D. XGBoost (Gradient Boosting Machine):

XGBoost trains decision trees sequentially. Each new tree focuses on correcting the gradient errors made by the previous trees.

The parameters were configured with 100 boosting rounds and a computed parameter scale_pos_weight set to the negative-to-positive ratio (~5.0).

Gradient boosting usually delivers high classification accuracy for tabular data. By setting scale_pos_weight, we explicitly direct the algorithm's loss function to penalize false negatives (missed fraud) five times more than false positives.

3. Core Evaluation Metrics

While standard classification accuracy is recorded, we evaluate model effectiveness using metrics more suited to fraud detection:

- **Recall ({TP}/{TP + FN}):** The percentage of actual fraud transactions caught. High recall minimizes financial exposure.
- **Precision ({TP}/{TP + FP}):** The percentage of predicted fraud transactions that are actually fraudulent. High precision minimizes "false alarm" friction for legitimate customers.
- **F1-Score (2 x {{Precision} x {Recall}}/{{Precision} + {Recall}}):** The harmonic mean of Precision and Recall. This serves as our single primary metric to balance risk management against customer experience.

Next Steps for the Team

This pipeline trains all four models, evaluates them on realistic data, and saves them locally as serialized .pkl files.

The deliverables will be passed to **Group C (Model Evaluation & Deployment)**, who will compare the final metrics, plot the ROC-AUC curves, analyze the confusion matrices, and integrate the champion model into the Streamlit application.